EEG Channel Selection and Model Compression for Motor Imagery BCI

Peiye Li | COMP 690-193, Special Topics in CS | Fall 2026

1 Problem and motivation

I propose a software study of how reducing EEG input channels interacts with model compression in a motor imagery brain-computer interface (BCI). Using EEGNet on BCI Competition IV Dataset 2a, I will test whether selecting electrodes for both task relevance and low feature redundancy preserves decoding accuracy across recording sessions while reducing computational and memory costs. All experiments will run on a personal computer.

The BCI-specific question is which recorded information can be removed without losing the ability to distinguish imagined movements. EEG electrodes are physical measurement locations, whereas CNN feature channels are learned internal representations. Reducing these two kinds of channels can have different effects. Electrode selection also changes the proposed sensing configuration; model pruning alone does not. The project will relate these choices to class-specific errors and variation among participants, rather than report compression ratios alone.

Primary research question: At a fixed EEG channel count, does accounting for redundancy in motor-related band-power features improve held-out-session accuracy over relevance-only and spatial-weight-based selection, and does any advantage survive INT8 quantization?

I hypothesize that a task-aware subset can retain complementary information more effectively than random selection, but that correlated channels are not necessarily dispensable. The study will test this hypothesis; improved accuracy or speed is not assumed.

2 Background and expected contribution

EEGNet uses depthwise and separable convolutions for compact EEG decoding [1]. Q-EEGNet already demonstrates 8-bit quantization and hardware-aware embedded implementation [3]. MI-BMInet combines spatial-filter-based EEG channel selection, quantization, memory analysis, and MCU deployment [4]. EEG-TCNet provides another compact MI decoding reference [5]. These studies establish substantial prior work on efficient BCI inference.

My contribution will be a controlled comparison within a fixed EEGNet pipeline: separating the effects of EEG electrode selection, redundancy-aware ranking, structured feature pruning, and numerical precision under a held-out-session protocol. The selection rule is an interpretable heuristic based on established feature-selection ideas, not a claim of a new compression algorithm. The outcome will be evidence about when these design choices preserve four-class MI information and when their apparent efficiency gains fail on the measured CPU.

3 Proposed approach

Baseline. Use the Braindecode EEGNet implementation in PyTorch and record its version, architecture, preprocessing, and training settings. Establish a full 22-channel FP32 baseline. Verify export accuracy before compression. Keep the decoder architecture and training budget fixed across channel-selection methods, changing only dimensions required by the selected inputs.

Task relevance and redundancy. For each electrode, compute trial-level log power in the 8-13 Hz and 13-30 Hz bands using training data only. Define relevance as the mean multiclass Fisher score across the two bands: between-class variation relative to within-class variation. Scale electrode relevance scores to the range 0-1. Define pairwise redundancy as the mean absolute Spearman correlation of their log-power features across training trials and the two bands. These are feature-level proxies, not proof of shared neural sources.

Selection rule. Start with the highest-relevance electrode. Repeatedly add the electrode maximizing its relevance minus 0.5 times its mean redundancy with the selected set. Keep 16 or 8 of the 22 electrodes. Fix the penalty before test evaluation; setting it to zero gives the relevance-only ablation. Fit every ranking inside the training portion of each validation split. Retrain EEGNet on each subset so the input tensor and spatial kernel actually shrink.

Controls. At each channel count, compare the proposed rule with relevance-only ranking, random subsets, and a spatial-weight-norm ranking inspired by MI-BMInet [4]. The latter ranks electrodes by the L2 norm of their learned EEGNet spatial weights and is an adaptation, not a reproduction of MI-BMInet. Use three predetermined random subsets and report all of them. Compare each method at equal channel count; include full-channel EEGNet as the reference.

Quantization. Export to ONNX Runtime and apply static INT8 post-training quantization using calibration samples drawn only from training data [6]. Evaluate both FP32 and quantized versions of the channel-selection conditions. Record which operations actually use integer kernels and which remain floating point. If quantization reduces validation accuracy by more than two percentage points, quantization-aware training (QAT) may be attempted for one finalist only, provided export is supported.

Structured pruning. As a secondary comparison, remove 25% or 50% of the output filters in EEGNet's final pointwise convolution using L1 ranking, update the associated normalization and classifier dimensions, and fine-tune with a fixed budget. Apply this to the full-channel model and one validation-selected channel-reduced model, followed by INT8. Removing tensor dimensions is required; zero masks alone do not establish speed or memory savings. EEG electrode count stays unchanged in the pruning-only condition.

Scope control. The minimum complete study is channel-selection controls crossed with FP32 and INT8. The bounded pruning comparison follows that study. INT4, extensive pruning sweeps, and a second neural recording modality are outside the required scope.

4 Evaluation plan and infrastructure

Data and preprocessing. Dataset 2a contains nine participants, two sessions on different days, and four MI classes: left hand, right hand, feet, and tongue [2]. Use the 22 EEG signals at 250 Hz, excluding EOG from decoder inputs. Keep the recorded reference and use a fixed causal 4-38 Hz filter. Analyze one two-second window per trial, from 0.5 to 2.5 seconds after cue onset, identically for all models. Reset filter state at run boundaries. Apply the same preprocessing in training and replay.

Separation of training and testing. For each participant, reserve the second session for final testing. Use three-fold validation grouped by entire runs within the first session for model selection and stopping decisions. Estimate normalization, channel rankings, and quantization calibration only from the applicable training partition. Then refit on session one with settings fixed by validation. Do not use session-two labels to select channels, compression levels, or checkpoints. Report a prespecified primary analysis on artifact-free trials and a secondary analysis including flagged trials; disclose exclusions and counts.

Decoding outcomes. Run final models with three training seeds. Report accuracy and macro-F1 by participant, class recall, and confusion matrices. Average seeds within participant before summarizing the nine participants; seeds and trials are not independent participant samples. Report paired accuracy differences with participant-level bootstrap intervals as exploratory uncertainty estimates. Show the number of participants losing more than two percentage points. Selected-electrode maps and validation-fold selection stability will support descriptive interpretation.

Efficiency and infrastructure. Measure all variants on the same CPU and ONNX Runtime configuration, batch size one and one inference thread. Record CPU model, operating system, runtime version, and power mode. After at least 100 warm-up calls, time at least 1,000 inferences over a fixed representative set of inputs; report median and 95th-percentile latency. Report preprocessing time separately. Compare serialized model size, parameter bytes, MACs, and input/activation tensor bytes under an explicit execution schedule. Any process-memory measurement includes runtime overhead and is not MCU SRAM usage.

Decision rule and claim boundaries. Plot accuracy against channel count, model size, and measured CPU latency. Select a final configuration using validation results: prioritize fewer electrodes among models within two percentage points of the full-channel FP32 baseline, then smaller model size. Report its test performance even if it misses this target. Keep observed test Pareto curves descriptive. No MCU board or simulator is required; desktop timing and tensor accounting cannot verify Cortex-M latency, peak SRAM, battery life, or energy use.

5 Deliverables and demonstration

Deliver a reproducible repository with fixed splits, configurations, seeds, channel lists, model exports, and benchmarking scripts; a report with ablations and participant-level results; and a final presentation. Replay recorded EEG at its original sampling rate through the frozen decoder, showing predicted classes, cursor movements, and compute timing. Label this a real-time replay demonstration: the recorded user cannot react to its feedback. Display the two-second observation window separately from compute latency; the primary decision is available 2.5 seconds after the cue plus processing time. Replay does not validate online closed-loop control or asynchronous intent detection.

6 Timeline

September: establish the data splits and baseline; verify CPU export and INT8 operator support on a pilot participant. Record pilot results as development evidence only.

Early to mid October: implement the channel-selection rules and controls; complete validation and select fixed settings. Track compute cost to keep the study feasible on a personal computer.

Late October to early November: complete FP32/INT8 comparisons and the bounded pruning study. Freeze the analysis plan and final configurations before inspecting held-out-session results.

November: evaluate all nine participants, produce uncertainty summaries and efficiency plots, and build the replay demonstration. Late November to early December: finalize the report and presentation according to the course deadline.

7 Risks and limits

Redundancy penalties may discard useful signals or fail across sessions. Compare directly with relevance-only and weight-based controls, report per-class failures, and retain negative results. Limited data may make selected electrodes unstable; quantify this rather than interpret a single map as a neural mechanism.

Quantized kernels may be unsupported or slower than FP32 on the available CPU. Check this early, document mixed-precision execution, and report measured outcomes. If computation is excessive, prioritize the complete channel-selection/INT8 study and reduce secondary pruning work; preserve the held-out test set.

The evidence will concern scalp EEG and within-participant transfer to another session. It will not establish transfer to new participants, other datasets, or implanted recordings. A similar evaluation framework could be reused for ECoG or intracortical signals, but the relevant features, spatial redundancy, and decoding task would need separate validation. The original intracortical GRU extension is deferred so that this distinction remains explicit.

8 References

[1] Lawhern, V. J., et al. EEGNet: A Compact Convolutional Neural Network for EEG-based Brain-Computer Interfaces. Journal of Neural Engineering, 15, 056013, 2018. https://doi.org/10.1088/1741-2552/aace8c

[2] Brunner, C., et al. BCI Competition 2008 - Graz data set A. Official Dataset 2a description, 2008. https://bbci.de/competition/iv/desc_2a.pdf

[3] Schneider, T., et al. Q-EEGNet: an Energy-Efficient 8-bit Quantized Parallel EEGNet Implementation for Edge Motor-Imagery Brain-Machine Interfaces. SMARTCOMP, 2020. https://arxiv.org/abs/2004.11690

[4] Wang, X., et al. MI-BMInet: An Efficient Convolutional Neural Network for Motor Imagery Brain-Machine Interfaces with EEG Channel Selection. arXiv:2203.14592, version 5, 2024. https://arxiv.org/abs/2203.14592v5

[5] Ingolfsson, T. M., et al. EEG-TCNet: An Accurate Temporal Convolutional Network for Embedded Motor-Imagery Brain-Machine Interfaces. IEEE SMC, 2020. https://arxiv.org/abs/2006.00622

[6] ONNX Runtime documentation. Quantize ONNX models. Accessed September 14, 2026. https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html